



PRÉ-RENTRÉE 2026-2027

# SÉANCE 4 - STRUCTURES DE DONNÉES ET CSV ROBUSTE

Dossier personnalisé - Ahmed BENHADJ SALEM - durée : 2 heures

Parcours : Fiabilisation et autonomie - candidat individuel

**Objectif personnel de la séance 4 :** maîtriser les effets de la mutabilité, transformer une table CSV brute en enregistrements fiables, isoler les lignes invalides et démontrer que les filtres et tris ne corrompent pas la table source.

**Production à conserver :** `S4_structures_csv_Ahmed.py`, capable de charger `mesures_capteurs.csv`, de signaler précisément les anomalies du fichier de test, de filtrer, trier et calculer des statistiques avec des tests explicites.

## Contrat de fiabilisation

1. Je distingue le **conteneur** des objets qu'il référence avant toute copie.
2. Je fixe le schéma et les contraintes d'une ligne avant de la convertir.
3. Une ligne invalide ne fait pas échouer tout le chargement : elle produit une erreur exploitable.
4. Je sépare données valides et journal d'erreurs.
5. Je préfère `sorted` à `list.sort()` lorsqu'il faut préserver l'ordre source.
6. J'écris les cas limites et invalides avant l'implémentation définitive.

## 1. Feuille de route des 120 minutes

| Temps       | Travail                                             | Preuve attendue                   |
|-------------|-----------------------------------------------------|-----------------------------------|
| 0–10 min    | Rituel : alias, copie superficielle et types CSV.   | Trois prévisions justifiées.      |
| 10–28 min   | Séquences, opérations mutantes et tuples de retour. | Audit d'opérations complété.      |
| 28–45 min   | Objets imbriqués, aliasing et copie indépendante.   | Source préservée par un test.     |
| 45–65 min   | Schéma d'enregistrement et contrat de validation.   | Matrice de sept cas préparée.     |
| 65–70 min   | Pause.                                              | –                                 |
| 70–88 min   | Chargeur robuste avec erreurs structurées.          | Valides et erreurs séparées.      |
| 88–100 min  | Filtre, statistiques et non-mutation.               | Tests normaux et vides.           |
| 100–112 min | Tri, doublons et cohérence du statut.               | Trois invariants vérifiés.        |
| 112–120 min | Revue, oral et exit ticket.                         | Pipeline expliqué en 60 secondes. |

**Critères de réussite :** aucune mutation cachée ; toutes les conversions protégées ; identifiants dupliqués détectés ; erreurs associées à leur ligne ; filtrage et tri produisant de nouvelles structures ; moyenne contrôlée avec `isclose` ; dix tests conservés.

## 2. Rituel de fiabilité - 10 min

**Consigne :** prévoir sans exécuter, puis justifier par un modèle d’objets ou par le contrat d’une bibliothèque.

### Rituel 1 - alias simple

```
a = [1, 2]
b = a
b.append(3)
print(a is b, a)
```

Sortie prévue : \_\_\_\_\_

Justification : \_\_\_\_\_

### Rituel 2 - copie superficielle d’objets imbriqués

```
source = [{"id": "C01", "temperature": 28.4}]
copie = source.copy()
copie[0]["temperature"] = 99.0
print(source[0]["temperature"])
```

Valeur prévue : \_\_\_\_\_ Nombre de listes : \_\_\_\_\_ Nombre de dictionnaires : \_\_\_\_\_

Pourquoi la copie ne suffit-elle pas ?  
\_\_\_\_\_  
\_\_\_\_\_

### Rituel 3 - contrat de csv.DictReader

Pour la ligne CSV C01,2026-08-24,28.4,60,OK, compléter avant tout calcul.

| Expression           | Type avant conversion | Type nécessaire |
|----------------------|-----------------------|-----------------|
| ligne["id"]          |                       |                 |
| ligne["temperature"] |                       |                 |
| ligne["humidite"]    |                       |                 |

Certitude : ☐ 1   ☐ 2   ☐ 3   ☐ 4 Contrôle : \_\_\_\_\_

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l’enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

### 3. Séquences, mutation et tuple de retour - 18 min

#### A. Auditer les opérations

Pour chaque expression, indiquer si elle modifie la liste source et ce qu'elle produit.

| Expression  | Source modifiée ? | Résultat / effet |
|-------------|-------------------|------------------|
| L.append(x) |                   |                  |
| L.sort()    |                   |                  |
| sorted(L)   |                   |                  |
| L + [x]     |                   |                  |
| L.copy()    |                   |                  |
| L[:]        |                   |                  |

#### B. Compréhension : filtrer en construisant

```
valeurs = [-2, 4, 0, 7, -1]
positives = [x for x in valeurs if x > 0]
```

1. Prévoir positives : \_\_\_\_\_
2. La liste valeurs est-elle modifiée ? Donner une preuve exécutable.  
\_\_\_\_\_
3. Réécrire la compréhension pour conserver les valeurs  $\geq 0$ .  
\_\_\_\_\_

#### C. Pourquoi un tuple en S4 ?

Le chargeur robuste renverra deux résultats : les lignes valides et les erreurs.

```
mesures, erreurs = charger_mesures_robuste(chemin)
```

1. Quel type peut regrouper ces deux valeurs renvoyées ? \_\_\_\_\_
2. Expliquer le déballage des deux variables.  
\_\_\_\_\_
3. Une clé de tri pourra être le tuple (temperature, id) : préciser l'ordre de comparaison utilisé si deux températures sont égales.  
\_\_\_\_\_

Aides graduées - une seule à la fois : A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 4. Aliasing imbriqué et copie réellement indépendante - 17 min

### A. Dessiner le graphe d'objets

Pour le programme ci-dessous, représenter les deux listes et les dictionnaires qu'elles référencent.

```
mesures = [  
    {"id": "C01", "temperature": 28.4},  
    {"id": "C02", "temperature": 31.2}  
]  
copie = mesures.copy()
```

### B. Trois stratégies à comparer

| Construction                                     | Nouvelle liste ? | Dictionnaires indépendants ? |
|--------------------------------------------------|------------------|------------------------------|
| <code>copie = mesures</code>                     |                  |                              |
| <code>copie = mesures.copy()</code>              |                  |                              |
| <code>copie = [m.copy() for m in mesures]</code> |                  |                              |

### C. Test de non-mutation

Compléter une preuve exécutable avant d'écrire le filtre définitif.

```
source = [{"id": "C01", "statut": "ALERTE"}]  
resultat = [m.copy() for m in source if m["statut"] == "ALERTE"]  
resultat[0]["statut"] = "TRAITEE"  
assert source[0]["statut"] == _____
```

**Point de méthode :** la question n'est pas seulement « les deux listes sont-elles différentes ? ». Il faut aussi vérifier si leurs éléments mutables sont encore partagés.

### D. Invariant du futur chargeur

Formuler en une phrase ce qui doit être vrai de chaque dictionnaire ajouté à `mesures_valides`.

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 5. Définir le schéma et les cas de validation - 20 min

Une ligne brute de DictReader contient des chaînes. Une ligne validée doit devenir un nouveau dictionnaire respectant un schéma, des types et des contraintes explicites.

| Clé         | Type après validation | Contrainte                                  |
|-------------|-----------------------|---------------------------------------------|
| id          | str                   | non vide et unique ;                        |
| date        | str                   | forme simple AAAA-MM-JJ ;                   |
| temperature | float                 | conversion possible ;                       |
| humidite    | int                   | comprise entre 0 et 100 ;                   |
| statut      | str                   | OK ou ALERTE, cohérent avec la température. |

### Contrat de valider\_ligne

```
def valider_ligne(ligne, ids_vus):  
    """Renvoie (mesure, None) si la ligne est valide,  
    sinon (None, raison). Ne modifie pas ligne."""
```

### Matrice de cas à fixer avant le code

Compléter **Valide ?** et une raison précise. Une ligne ne doit pas être acceptée par hasard après une exception.

| N° | Modification de la ligne de référence | Valide ? | Raison attendue |
|----|---------------------------------------|----------|-----------------|
| 1  | aucune modification                   |          |                 |
| 2  | identifiant vide                      |          |                 |
| 3  | température "abc"                     |          |                 |
| 4  | humidité "120"                        |          |                 |
| 5  | statut "INCONNU"                      |          |                 |
| 6  | identifiant déjà présent dans ids_vus |          |                 |
| 7  | température -2.0 mais statut OK       |          |                 |

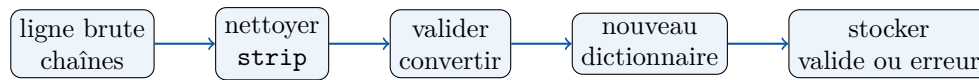
**Preuve avant la pause :** les sept décisions sont écrites ; le code devra respecter ces décisions et non l'inverse.

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 6. Construire un chargeur CSV robuste - 18 min

Fichiers : mesures\_capteurs.csv puis mesures\_capteurs\_anomalies.csv.

### A. Pipeline imposé



### B. Compléter le squelette sans mélanger les responsabilités

```

def charger_mesures_robuste(chemin):
    mesures_valides = []
    erreurs = []
    ids_vus = {}
    with open(chemin, newline="", encoding="utf-8") as fichier:
        lecteur = csv.DictReader(fichier)
        for numero, ligne in enumerate(lecteur, start=2):
            mesure, erreur = _____
            if erreur is None:
                _____
                ids_vus[mesure["id"]] = numero
            else:
                erreurs.append({
                    "ligne": numero,
                    "id": ligne.get("id", ""),
                    "raison": _____
                })
    return mesures_valides, erreurs
  
```

### C. Décisions d'architecture

1. Pourquoi ids\_vus est-il un dictionnaire plutôt qu'un simple booléen ?  
\_\_\_\_\_
2. Pourquoi stocker le numéro de ligne dans le journal d'erreurs ?  
\_\_\_\_\_
3. Pourquoi la conversion doit-elle être protégée avant d'ajouter la mesure ?  
\_\_\_\_\_

**Interdit :** corriger silencieusement une valeur invalide par une valeur arbitraire. Le programme doit soit produire une mesure conforme au contrat, soit expliquer le rejet.

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 7. Filtrer, calculer et prouver la non-mutation - 12 min

### A. Filtre indépendant

Compléter dans `S4_structures_csv_Ahmed.py` :

```
def filtrer_alertes(mesures):  
    return [_____]
```

Contraintes : compréhension ; critère explicite ; chaque dictionnaire retenu doit être indépendant de l'original.

### B. Statistiques avec contrat vide

```
def statistiques_temperatures(mesures):  
    """Renvoi (minimum, maximum, moyenne), ou None si vide."""  
    # implementation a construire
```

| N° | Cas                    | Résultat attendu | Contrôle            |
|----|------------------------|------------------|---------------------|
| 1  | liste vide             |                  | contrat explicite   |
| 2  | une seule température  |                  | min = max = moyenne |
| 3  | plusieurs températures |                  | min ≤ moyenne ≤ max |
| 4  | moyenne flottante      |                  | isclose             |

### C. Test d'indépendance après filtrage

Écrire un test qui :

1. crée une liste source contenant une alerte ;
2. appelle `filtrer_alertes` ;
3. modifie le statut dans le résultat ;
4. vérifie que le statut de la source n'a pas changé.

**Continuité avec S3 :** le comportement vide, les types de sortie et les cas limites sont des éléments du contrat, pas des détails à décider après l'échec d'un test.

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 8. Trier, détecter les doublons et contrôler la cohérence - 12 min

### A. Tri stable sans mutation

```
def cle_tri(mesure):  
    return (mesure["temperature"], mesure["id"])  
  
def trier_mesures(mesures):  
    return sorted(mesures, key=cle_tri)
```

- 1. Pourquoi le second élément du tuple est-il utile ?
- 2. Écrire un test qui conserve l'ordre initial dans une variable, appelle le tri, puis vérifie que `mesures` n'a pas changé.

### B. Doublet : localiser et expliquer

Le dictionnaire `ids_vus` associe un identifiant à sa première ligne valide.

- 1. Quelle information doit contenir l'erreur lorsqu'un identifiant est revu ?
- 2. Un identifiant invalide doit-il être ajouté à `ids_vus` ? Justifier.

### C. Cohérence croisée avec la séance 1

Règle du stage : ALERTE si  $t < 0$  ou  $t > 30$ , sinon OK.

```
def statut_attendu(temperature):  
    -----
```

Écrire deux cas de frontière et deux cas extérieurs avant de coder.

| N° | Température | Statut attendu | Test écrit               |
|----|-------------|----------------|--------------------------|
| 1  |             |                | <input type="checkbox"/> |
| 2  |             |                | <input type="checkbox"/> |
| 3  |             |                | <input type="checkbox"/> |
| 4  |             |                | <input type="checkbox"/> |

**Aides graduées - une seule à la fois :** A - dessiner les objets mutables partagés ; B - écrire le schéma de l'enregistrement avant le code ; C - séparer conversion, validation et stockage ; D - construire un contre-exemple de ligne invalide ; E - vérifier non-mutation, doublons, types et cohérence croisée.

## 9. Atelier intégré : valides, erreurs, rapport - 18 min

**Fichier :** S4\_structures\_csv\_Ahmed.py. Le résultat final doit fonctionner sur le fichier propre et sur le fichier d'anomalies sans modifier les données brutes ni masquer les erreurs.

### Ordre d'implémentation imposé

| Étape | Production                        | Preuve avant passage                     |
|-------|-----------------------------------|------------------------------------------|
| 1     | date_iso_simple et statut_attendu | bornes et formats testés ;               |
| 2     | valider_ligne                     | sept cas de la matrice ;                 |
| 3     | charger_mesures_robuste           | erreurs avec numéros de ligne ;          |
| 4     | filtrer_alertes                   | source préservée ;                       |
| 5     | statistiques_temperatures         | vide, singleton, flottant ;              |
| 6     | trier_mesures                     | ordre source préservé ;                  |
| 7     | rapport final                     | valides, erreurs, alertes, statistiques. |

### Matrice minimale de dix tests

| N° | Propriété testée               | Résultat attendu | Vert                     |
|----|--------------------------------|------------------|--------------------------|
| 1  | ligne conforme                 |                  | <input type="checkbox"/> |
| 2  | identifiant vide               |                  | <input type="checkbox"/> |
| 3  | température invalide           |                  | <input type="checkbox"/> |
| 4  | humidité hors intervalle       |                  | <input type="checkbox"/> |
| 5  | statut inconnu                 |                  | <input type="checkbox"/> |
| 6  | doublon                        |                  | <input type="checkbox"/> |
| 7  | incohérence température/statut |                  | <input type="checkbox"/> |
| 8  | filtre indépendant             |                  | <input type="checkbox"/> |
| 9  | tri non mutant                 |                  | <input type="checkbox"/> |
| 10 | moyenne avec isclose           |                  | <input type="checkbox"/> |

### Journal de débogage et gestion du temps

| Minute | Premier symptôme | Modification unique | Régression ? |
|--------|------------------|---------------------|--------------|
|--------|------------------|---------------------|--------------|

**Seuil de décision :** si un bug persiste après deux modifications, réduire le cas au plus petit dictionnaire qui reproduit l'erreur, puis relancer uniquement le test concerné.

## 10. Cartes de rappel, oral et exit ticket - 8 min

### Carte A - Mutabilité

`append` et `sort` modifient. `sorted`, une compréhension et une copie construisent un nouvel objet, mais une copie superficielle peut encore partager ses éléments.

### Carte B - Schéma

Une ligne valide possède les clés attendues, les bons types et des contraintes cohérentes. Le schéma est écrit avant le chargeur.

### Carte C - CSV robuste

Lire brut, nettoyer, convertir, valider, créer un nouveau dictionnaire, puis stocker dans `valides` ou `erreurs`.

### Carte D - Preuves

Cas normal, frontière, invalide, doublon, non-mutation, contrat vide et flottant avec `isclose`.

### Oral de 60 secondes - sans lire le code

Présenter le pipeline en utilisant les six mots : **ligne brute**, **conversion**, **validation**, **enregistrement**, **filtre**, **preuve**.

### Exit ticket - sans aide

1. Pourquoi `liste.copy()` peut-elle être insuffisante pour une liste de dictionnaires ?

2. Quel type initial reçoit-on pour une température lue par `DictReader` ? \_\_\_\_\_

3. Pourquoi préférer `sorted` si la fonction promet de ne pas modifier son argument ?

4. Donner un cas où la ligne est syntaxiquement convertible mais incohérente.

5. Citer la preuve de robustesse la plus convaincante obtenue aujourd'hui.

**Fichiers à conserver pour S5 :** `S4_structures_csv_Ahmed.py`, `mesures_capteurs.csv`, `mesures_capteurs_anomalies.csv` et le journal des tests. Le mini-projet final devra réutiliser uniquement les fonctions déjà validées.